

Ollama 설치 순서

1. Ollama 설치
2. 32k 컨텍스트 모델 생성

1. Ollama설치

윈도우에 Ollama설치 : <https://ollama.com/>

칼리에 깔아서 해봤는데, 모델이 너무 커서 vm환경에서 병목현상이 일어납니다.

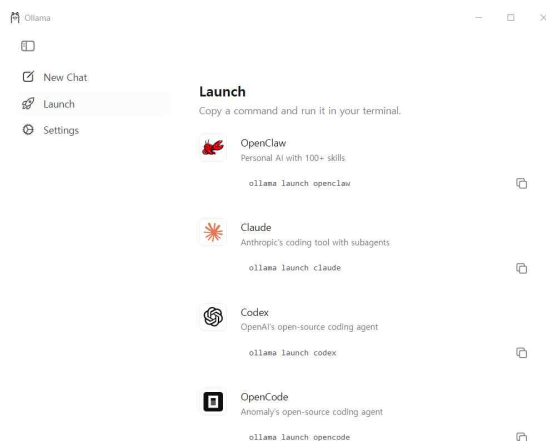
윈도우 호스트로 Ollama를 따로 vmware 밖으로 빼서 사용하겠습니다.

- 추론: Windows 호스트의 Ollama + GPU
- 실행: Kali 내부 bash 스크립트
- 결과 저장: Kali 내부 bash 또는 Python
- AI 요약: Ollama API 호출

구조

- Windows 호스트: Ollama 설치 및 모델 실행
- Kali VM: 자동화 스크립트 실행
- Kali -> Windows Ollama API 호출
- 결과를 audit-report.md로 저장

[Ollama 런처]



2. 32k 컨텍스트 모델 생성

코딩/에이전트용 모델 : **qwen2.5-coder:7b**

powershell (x86)

```
ollama pull qwen2.5-coder:7b
```

```
PS C:\WINDOWS\system32> ollama pull qwen2.5-coder:7b
pulling manifest
pulling 60e05f210007: 66% | 3.1 GB/4.7 GB 89 MB/s 17s|
```

powershell (x86)

```
cd $HOME
mkdir ollama-work -Force
cd .\ollama-work
@"
FROM qwen2.5-coder:7b
PARAMETER num_ctx 32768
"@ | Set-Content -Encoding ascii .\Modelfile
```

Get-Content .\Modelfile

ollama create qwen2.5-coder-32k -f .\Modelfile

```
PS C:\WINDOWS\system32> cd $HOME
PS C:\Users\user> mkdir ollama-work -Force

디렉터리 : C:\Users\user

Mode                LastWriteTime         Length Name
----                -
d-----          2026-04-12 오후 1:16                ollama-work

PS C:\Users\user> cd .\ollama-work
PS C:\Users\user\ollama-work> @"
>> FROM qwen2.5-coder:7b
>> PARAMETER num_ctx 32768
>> "@ | Set-Content -Encoding ascii .\Modelfile
PS C:\Users\user\ollama-work>
PS C:\Users\user\ollama-work> Get-Content .\Modelfile
FROM qwen2.5-coder:7b
PARAMETER num_ctx 32768
PS C:\Users\user\ollama-work> ollama create qwen2.5-coder-32k -f .\Modelfile
gathering model components
using existing layer sha256:60e05f2100071479f596b964f89f510f057ce397ea22f2833a0cfe029bfc2463
using existing layer sha256:66b9ea09bd5b7099cbb4fc820f31b575c0366fa439b08245566692c6784e281e
using existing layer sha256:1e65450c30670713aa47fe23e8b9662bdf4065e81cc8e3cbfaa98924fcc0d320
using existing layer sha256:832dd9e00a68dd83b3c3fb9f5588dad7dcf337a0db50f7d9483f310cd292e92e
creating new layer sha256:db8fbfd0cb288a053f83ac9014ca9bac2558b1bbcd80b5c408a548e7acba8a24
writing manifest
success
PS C:\Users\user\ollama-work>
```

[모델 실행]

ollama run qwen2.5-coder-32k

```
PS C:\Users\user\ollama-work> ollama run qwen2.5-coder-32k
>>>
>>> Send a message (? for help)
```

*프롬프트 준비상태

ollama ps (GPU 사용 가능한지 확인)

```
PS C:\Users\user\ollama-work> ollama ps
NAME                ID                SIZE    PROCESSOR    CONTEXT    UNTIL
qwen2.5-coder-32k:latest  cf0e39a496ed    8.2 GB    100% GPU    32768      4 minutes from now
PS C:\Users\user\ollama-work>
```

window Ollama + kali 연결

setx OLLAMA_HOST "0.0.0.0:11434"

```
PS C:\Users\user\ollama-work> setx OLLAMA_HOST "0.0.0.0:11434"
```

성공 : 지정한 값을 저장했습니다.

설정 후 Ollama 완전히 종료 후 재실행

그러면 네트워크 설정 권한 요청이 오는데, 허용

이후 칼리와 연결할 IP를 확인 (NAT 대역 확인)

저같은 경우 kali : 192.168.152.128 이므로 152.x 대역을 확인

이더넷 어댑터 VMware Network Adapter VMnet8:

```
연결 별 DNS 접미사 . . . . . :
링크-로컬 IPv6 주소 . . . . . : fe80::cc5a:cb80:bc27:5be6%9
IPv4 주소 . . . . . : 192.168.152.1
서브넷 마스크 . . . . . : 255.255.255.0
기본 게이트웨이 . . . . . :
```

```
kali@kali: ~
파일 동작 편집하기 보기 도움말

valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel sta
te UP group default qlen 1000
    link/ether 00:0c:29:fd:57:76 brd ff:ff:ff:ff:ff:ff
    inet 192.168.152.128/24 brd 192.168.152.255 scope global dynamic n
oprefixroute eth0
        valid_lft 994sec preferred_lft 994sec
        inet6 fe80::20c:29ff:fe57:5776/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

[칼리 터미널]

curl http://윈도우 호스트IP:11434/api/tags

```
(kali㉿kali)-[~]
$ curl http://192.168.152.1:11434/api/tags
{"models":[{"name":"qwen2.5-coder-32k:latest","model":"qwen2.5-coder-32k:latest","modified_at":"2026-04-12T13:16:34.6538944+09:00","size":4683087653,"digest":"cf0e39a496ed6a66b219be2d822bf4380478fc6a15306c0a72d7472580c60d47","details":{"parent_model":"","format":"gguf","family":"qwen2","families":["qwen2"],"parameter_size":"7.6B","quantization_level":"Q4_K_M"}},{name":"qwen2.5-coder:7b","model":"qwen2.5-coder:7b","modified_at":"2026-04-12T13:12:41.1136605+09:00","size":4683087561,"digest":"dae161e27b0e90dd1856c8bb3209201fd6736d8eb66298e75ed87571486f4364","details":{"parent_model":"","format":"gguf","family":"qwen2","families":["qwen2"],"parameter_size":"7.6B","quantization_level":"Q4_K_M"}}]}
```

curl http://윈도우 호스트IP:11434/v1/models

```
(kali㉿kali)-[~]
$ curl http://192.168.152.1:11434/v1/models
{"object":"list","data":[{"id":"qwen2.5-coder-32k:latest","object":"model","created":1775967394,"owned_by":"library"}, {"id":"qwen2.5-coder:7b","object":"model","created":1775967161,"owned_by":"library"}]}
```

****이렇게 뜨면 정상적으로 연결 완료****

[칼리 터미널]

nano ~/.config/opcode/opcode.json

```
kali@kali: ~
파일 동작 편집하기 보기 도움말
GNU nano 8.3 /home/kali/.config/opcode/opcode.json
1
{"schema": "https://opcode.ai/config.json",
  "provider": {
    "ollama": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "Ollama (Windows host)",
      "options": {
        "baseUrl": "http://192.168.152.1:11434/v1"
      }
    }
  },
  "models": {
    "qwen2.5-coder-32k": {
      "name": "qwen2.5-coder-32k"
    }
  }
},
  "model": "ollama/qwen2.5-coder-32k",
  "small_model": "ollama/qwen2.5-coder-32k",
```

아래 내용으로 덮어쓰기

```
-----
{
```

```
"$schema": "https://opencode.ai/config.json",
"provider": {
  "ollama": {
    "npm": "@ai-sdk/openai-compatible",
    "name": "Ollama (Windows host)",
    "options": {
      "baseUrl": "http://192.168.152.1:11434/v1"
    },
    "models": {
      "qwen2.5-coder-32k": {
        "name": "qwen2.5-coder-32k"
      }
    }
  }
},
"model": "ollama/qwen2.5-coder-32k",
"small_model": "ollama/qwen2.5-coder-32k",
"permission": {
  "bash": "ask",
  "edit": "ask"
}
}
```

이후 작업폴더 다시 실행

cd ~/agent-lab

[작업폴더 만들기]

mkdir -p ~/agent-lab/{scripts,.opencode/commands}

cd ~/agent-lab



```
(kali@kali)-[~]
└─$ mkdir -p ~/agent-lab/{scripts,.opencode/commands}
cd ~/agent-lab
```

자동화 검증 방식

기존 OpenCode 중심 검증 대신, 아래 흐름으로 실제 동작을 검증합니다.

- 1) collect.sh 실행
- 2) 결과를 collect.out으로 저장
- 3) Ollama API에 collect.out 내용을 보내 요약 생성
- 4) audit-report.md 파일 저장

[테스트형 자동화 쉘 스크립트 실행]

첨부된 agent_test.sh을 /agent-lab에 넣고 아래 명령어 실행

```
bash ./agent_test.sh /home/kali/agent-lab http://윈도우호스트IP:11434 qwen2.5-coder-32k
```

```
(kali㉿kali)-[~/agent-lab]
$ bash ./agent_test.sh /home/kali/agent-lab http://192.168.152.1:11434 qwen2.5-coder-32k
4 qwen2.5-coder-32k
[*] Running: /home/kali/agent-lab/scripts/collect.sh
[*] Done
    collect.out      → /home/kali/agent-lab/collect.out
    audit-report.md  → /home/kali/agent-lab/audit-report.md

——— audit-report.md preview ———
# Audit Report

- execution time: 2026-04-12 14:30:08 KST
- commands executed:
  - date
  - whoami
  - pwd
  - ls
  - uname -a
- result summary:
  - Current directory is `~/agent-lab`
  - User running the script is `kali`
  - Script is running on a Kali Linux system with kernel version `6.12.13-amd64`
- issues if any: 특이사항 없음
```

collect.out 파일과 audit-report.md 파일이 자동으로 생성됨.

```
(kali㉿kali)-[~/agent-lab]
$ ls
AGENTS.md      audit-report.md  run_time.txt  test.txt
agent_test.sh  collect.out      scripts
```